

School of Computer Science

Introduction to Fuzzing

Hastily covered by Gretch Hallett
Email: gretch.hallett@bristol.ac.uk

~~Dr. Sana Belguith~~

~~Sana.belguith@bristol.ac.uk~~

bristol.ac.uk

Fuzzing

- No, it is not about Fuzzy logic
- Neither about fuzzy set membership

Security Software Testing

Memory Corruption Bugs

Exploitable

References:

1. book (Chapter 1, section 1.3): *Fuzzing for Software Security Testing and Quality Assurance.* Ari Takanen, Jared DeMott, Charlie Miller

2. Article *"Fuzzing: Hack, Art, and Science"* By P. Godefroid

How do we track how many bugs we have?

1. All software is buggy
2. We try and fix the bugs
3. It would be nice if we could tell people which updates fix which bugs, and which bugs are being actively exploited and share…

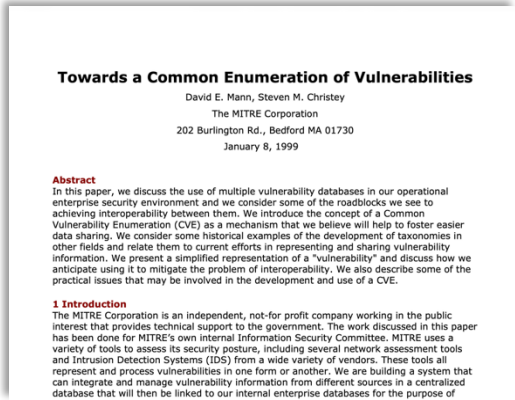
If *only* we had some sort global bug database for security vulnerabilities…

CVE

Common Vulnerabilities Enumeration

- Back In 1999 David Mann and Steven Christey at MITRE/Purdue Uni write a paper and publish the first list
- Companies can apply to become CVE Numbering Authorities (CNAs) and assign bugs numbers for some software (e.g. Linux Foundation for Linux)

- Security researchers start to judge themselves by how many CVEs they can get assigned
  - (this is stupid)



Towards a Common Enumeration of Vulnerabilities

David E. Mann, Steven M. Christey
The MITRE Corporation
202 Burlington Rd., Bedford MA 01730
January 8, 1999

Abstract
In this paper, we discuss the use of multiple vulnerability databases in our operational enterprise security environment and we consider some of the roadblocks we see to achieving interoperability between them. We introduce the concept of a Common Vulnerability Enumeration (CVE) as a mechanism that we believe will help to foster easier data sharing. We consider some historical examples of the development of taxonomies in other fields and relate them to current efforts in representing and sharing vulnerability information. We present a simplified representation of a "vulnerability" and discuss how we anticipate using it to mitigate the problem of interoperability. We also describe some of the practical issues that may be involved in the development and use of a CVE.

1 Introduction
The MITRE Corporation is an independent, not-for profit company working in the public interest that provides technical support to the government. The work discussed in this paper has been done for MITRE's own internal Information Security Committee. MITRE uses a variety of tools to assess its security posture, including several network assessment tools and Intrusion Detection Systems (IDS) from a wide variety of vendors. These tools all represent and process vulnerabilities in one form or another. We are building a system that can integrate and manage vulnerability information from different sources in a centralized database that will then be linked to our internal enterprise databases for the purpose of

CNA: Debian GNU/Linux

Updated: 2021-11-05
Published: 2014-09-24

Description

GNU Bash through 4.3 processes trailing strings after function definitions in the values of environment variables, which allows remote attackers to execute arbitrary code via a crafted environment, as demonstrated by vectors involving the ForceCommand feature in OpenSSH sshd, the mod_cgi and mod_cgid modules in the Apache HTTP Server, scripts executed by unspecified DHCP clients, and other situations in which setting the environment occurs across a privilege boundary from Bash execution, aka "ShellShock." NOTE: the original fix for this issue was incorrect; CVE-2014-7169 has been assigned to cover the vulnerability that is still present after the incorrect fix.

Product Status

Learn more

Information not provided

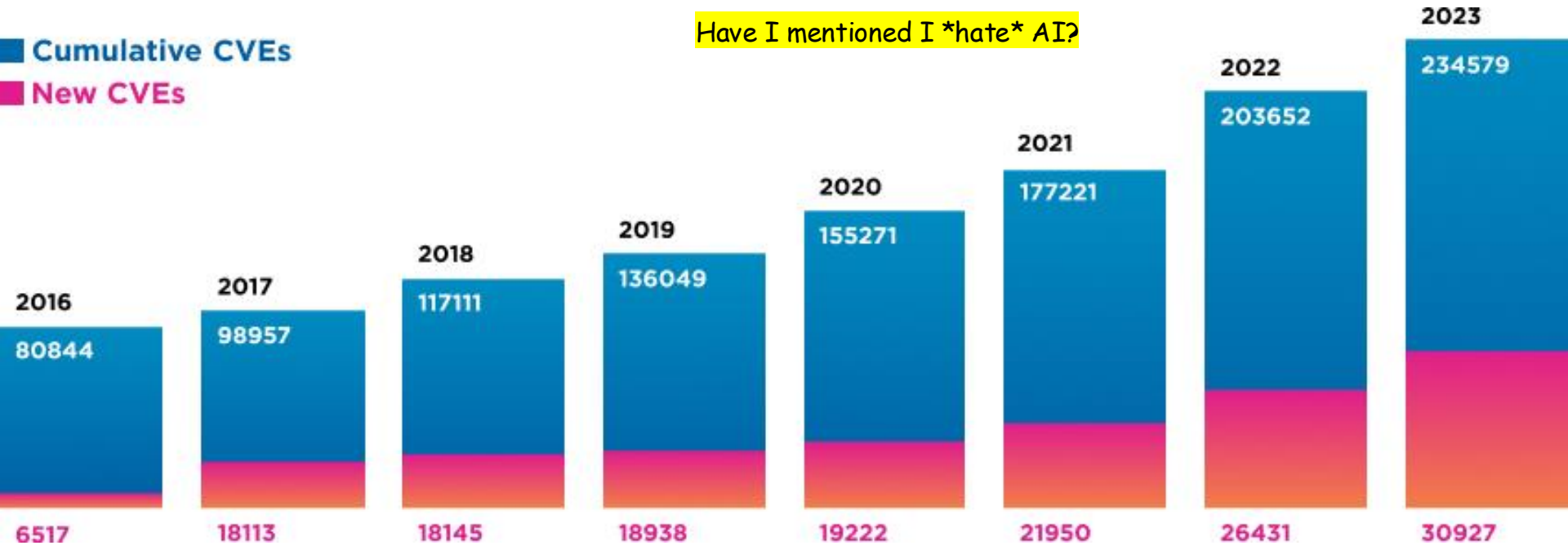
References 177 Total

- exploit-db.com: 37816 exploit
- http://packetstormsecurity.com/files/128517/VMware-Security-Advisory-2014-0010.html
- lists.opensuse.org: SUSE-SU-2014:1223 vendor-advisory
- http://www-01.ibm.com/support/docview.wss?uid=ssg1S1004897
- http://www-01.ibm.com/support/docview.wss?uid=swg21685749
- marc.info: HPSBMU03165 vendor-advisory
- marc.info: SSRT101816 vendor-advisory
- exploit-db.com: 39918 exploit

Why do we care?

As of 2024 it got SO MUCH WORSE: 40k CVEs then 48k CVEs last year, because AI has started applying for them (and some CNAs will give you a CVE for ANYTHING…)

Have I mentioned I *hate* AI?



https://www.skyboxsecurity.com/resources/report/vulnerability-threat-trends-report-2024/

The Plan

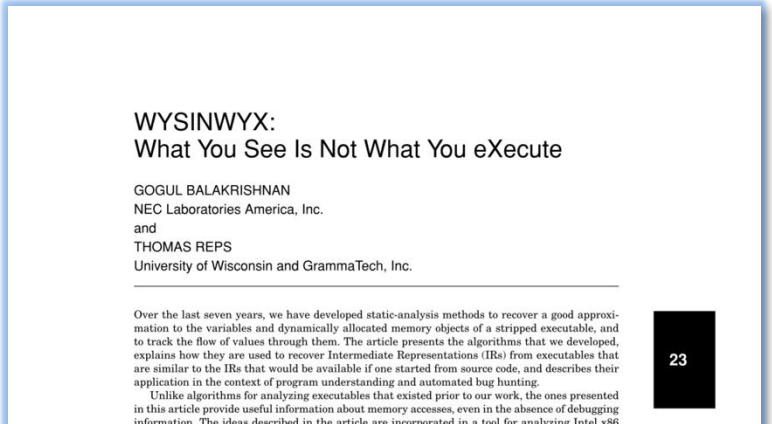
- Memory corruption vulnerabilities
- Fuzzing- finding vulnerabilities
- Types of Fuzzing
- Some existing solutions

Memory Corruption Vulnerabilities

- WYSINWYX: What You See Is Not What You eXecute by G. Balakrishnan et. al.
  - Higher level code -> low-level representation
  - Seemingly separate variables -> contiguous memory addresses

- Contiguous memory locations allow for boundary violations!

I kinda low-key hate this paper… it makes it seem like they invented hacking when less academic folk have been doing this for 30 years prior… — Gretch



WYSINWYX:
What You See Is Not What You eXecute

GOGUL BALAKRISHNAN
NEC Laboratories America, Inc.
and
THOMAS REPS
University of Wisconsin and GrammaTech, Inc.

Over the last seven years, we have developed static-analysis methods to recover a good approximation to the variables and dynamically allocated memory objects of a stripped executable, and to track the flow of values through them. The article presents the algorithms that we developed, explains how they are used to recover Intermediate Representations (IRs) from executables that are similar to the IRs that would be available if one started from source code, and describes their application in the context of program understanding and automated bug hunting.

Unlike algorithms for analyzing executables that existed prior to our work, the ones presented in this article provide useful information about memory accesses, even in the absence of debugging

Example

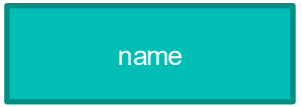
```
#include <stdio.h>
int get_cookie(){
  return rand();
}

int main(){
   int cookie;
   char name[40];
   cookie = get_cookie();
   gets(name);
   if (cookie == 0x41424344)
     printf("You win %s\n!", name);
   else
     printf("better luck next time\n");
   return 0;
}
```

Example

```
#include <stdio.h>
int get_cookie(){
  return rand();}
int main(){
    int cookie;
    char name[40];
    cookie = get_cookie();
    gets(name);
    if (cookie == 0x41424344)
        printf("You win %s\n!", name);
    else printf("better luck next time :(");
    return 0;

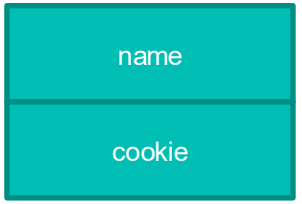
}
```



Example

```
#include <stdio.h>
int get_cookie(){
  return rand();}
int main(){
    int cookie;
    char name[40];
    cookie = get_cookie();
    gets(name);
    if (cookie == 0x41424344)
        printf("You win %s\n!", name);
    else printf("better luck next time :(");
    return 0;

}
```



Example

```
#include <stdio.h>
int get_cookie(){
  return rand();}
int main(){
    int cookie;
    char name[40];
    cookie = get_cookie();
    gets(name);
    if (cookie == 0x41424344)
        printf("You win %s\n!", name);
    else printf("better luck next time :(");
    return 0;

}
```

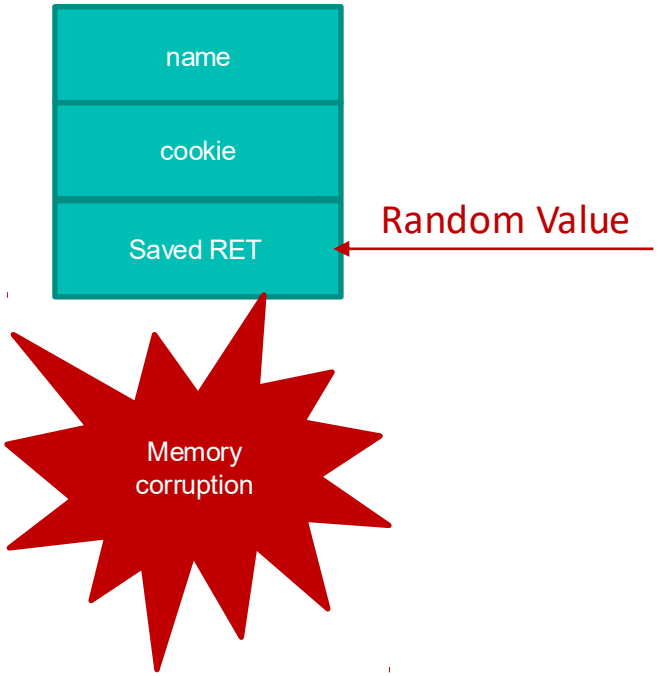


Buffer Overflow!

Example

```
#include <stdio.h>
int get_cookie(){
  return rand();}
int main(){
    int cookie;
    char name[40];
    cookie = get_cookie();
    gets(name);
    if (cookie == 0x41424344)
        printf("You win %s\n!", name);
    else printf("better luck next time :(");
    return 0;

}
```



Side effects

- Over/underflow
- Sensitive data corruption
- Control data corruption (control hijacking)

Side effects

- Over/underflow
- Sensitive data corruption
- Control data corruption (control hijacking)

If exploit is done properly, Otherwise crash!

Fuzzing

- *It started on a dark and stormy night…. [Barton P. Miller, late1980s]*



https://pages.cs.wisc.edu/~bart/fuzz/Foreword1.html

Sitting in my apartment in Madison in the Fall of 1988, there was a wild midwest thunderstorm pouring rain and lighting up the late night sky. He was logged on to the Unix system in my office via a dial-up phone line over a 1200 baud modem. With the heavy rain, there was noise on the line and that noise was interfering with my ability to type sensible commands to the shell and programs that I was running. It was a race to type an input line before the noise overwhelmed the command. This fighting with the noisy phone line was not surprising. After all, this was *just* before error-correcting modems were available.

What did surprise me was the fact that the noise seemed to be causing programs to crash. And more surprising to me were the programs that were crashing -- common Unix utilities that we all use everyday. The scientist in me said that we need to make a systematic investigation to try to understand the extent of the problem and the cause.

That semester, I was teaching the graduate Advanced Operating Systems course at the University of Wisconsin. Each semester in this course, we hand out a list of suggested topics for the students to explore for their course project. I added this testing project to the list; the text for this project description appears in the break-out box below. In the process of writing the project description, I needed to give this kind of testing a name. I wanted a name that would evoke the feeling of random, unstructured data. After trying out several ideas, I settled on the term ***fuzz***.

Three groups attempted the fuzz project that semester and two failed to achieve any crash results. Lars Fredriksen and Bryan So formed the third group, and were more talented programmers and more careful experimenters; they succeeded well beyond my expectations. As we reported in the first fuzz paper [1990], they could crash or hang between 25-33% of the utility programs on the seven Unix variants that they tested.

Fuzzing

- Security testing technique
- Run program on many **abnormal/malformed** inputs, look for **unintended** behavior, e.g. crash.
  - An observable (measurable) side effect is essential
  - Should be scalable



- Underlying assumption: *if the unintended behavior is dependent on input, an attacker can craft such an input to exploit the bug.*

Types of Fuzzing

- Input based: mutational and Generative (grammar based)
- Application based: black-box and white-box
- Input Strategy: memory-less and evolutionary

Input Generation

- Mutation Based: mutate seed inputs to create new test inputs:
  Simple strategy is to randomly choose an offset and change the byte.

Pros: easy to implement and low overhead

Cons: highly structured inputs will become invalid quickly → low coverage.

JPEG file format

Every week someone finds a new vulnerability in JPEG decoders it seems… lots issued last year…

JFIF file structure		
Segment	**Code**	**Description**
SOI	`FF D8`	Start of Image
JFIF-APP0	`FF E0 s1 s2 4A 46 49 46 00 ...`	see below
JFXX-APP0	`FF E0 s1 s2 4A 46 58 58 00 ...`	optional, see below
... additional marker segments (for example SOF, DHT, COM)		
SOS	`FF DA`	Start of Scan
	compressed image data	
EOI	`FF D9`	End of Image

JPEG file format

JFIF APP0 marker segment		
Field	**Size (bytes)**	**Description**
APP0 marker	2	`FF E0`
Length	2	Length of segment excluding APP0 marker
Identifier	5	`4A 46 49 46 00` = "JFIF" in ASCII, terminated by a null byte
JFIF version	2	First byte for major version, second byte for minor version (`01 02` for 1.02)
Density units	1	Units for the following pixel density fields • `00` : No units; width:height pixel aspect ratio = Ydensity:Xdensity • `01` : Pixels per inch (2.54 cm) • `02` : Pixels per centimeter
Xdensity	2	Horizontal pixel density. Must not be zero
Ydensity	2	Vertical pixel density. Must not be zero
Xthumbnail	1	Horizontal pixel count of the following embedded RGB thumbnail. May be zero
Ythumbnail	1	Vertical pixel count of the following embedded RGB thumbnail. May be zero

JFIF file structure		
Segment	**Code**	**Description**
SOI	`FF D8`	Start of Image
JFIF-APP0	`FF E0` *s1 s2* `4A 46 49 46 00 ...`	see below
JFXX-APP0	`FF E0` *s1 s2* `4A 46 58 58 00 ...`	optional, see below
... additional marker segments (for example SOF, DHT, COM)		
SOS	`FF DA`	Start of Scan
	compressed image data	
EOI	`FF D9`	End of Image

Input Generation

- Generation (Grammar) Based:
  - learn/create the format/model of the input
  - generate new inputs based on the learned model

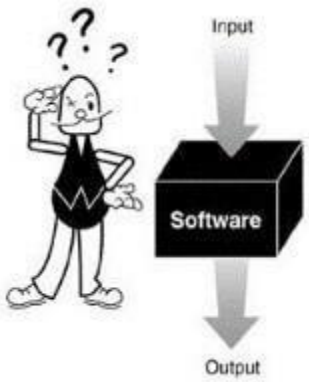
  e.g. well-known file formats (jpeg, xml, etc.)

Pros: Highly effective for complex structured input parsing applications → high coverage

Cons: expensive as models are not easy to learn or obtain.

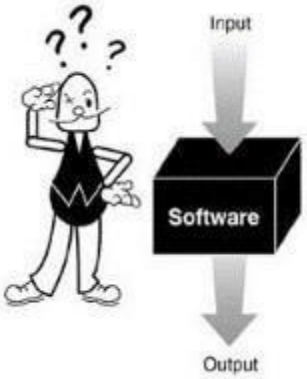
Application Monitoring

- Blackbox: Only interface is known.

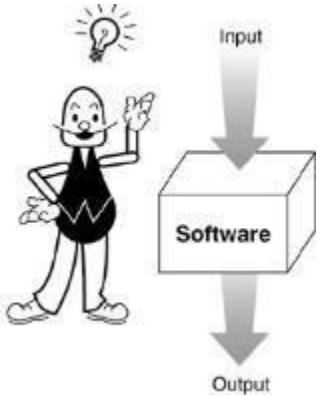


Application Monitoring

- Blackbox: Only interface is known.



- Whitebox: Application can be analysed/monitored.



Problem with Traditional Fuzzing

Blackbox + mutation: Aiming with luck!

```
... //JPEG parsing
read(fd, buf, size);
if (buf[1] == 0xD8 && buf[0] == 0xFF)
  // interesting code here
else
  pr_exit("Invalid file");
```



Problem with Traditional Fuzzing

- Apply more heuristics to:
  - Mutate better
  - Learn good inputs

- Apply more analysis (static/dynamic) to understand the application behavior.

  ➢ But remember the scalability factor!

Problem with ~~Traditional~~ Smart Fuzzing

smart fuzzing: Aiming with educated guess!



Evolutionary Fuzzing

- Rather than throwing inputs, *evolve them.*

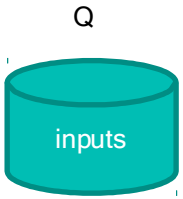
Evolve is, perhaps, the wrong word… we want to find the pathways through the program that lead to crashes… and then tweak them to see if we can get worse crashes.

Evolution of inputs that lead to death

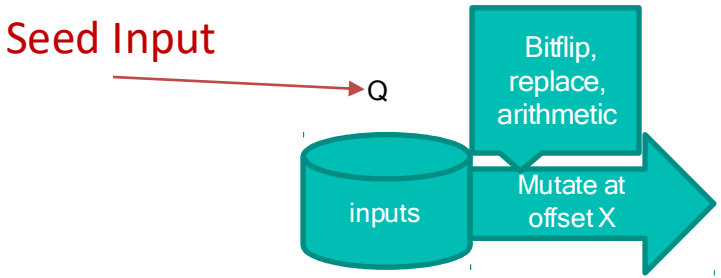
- *Underlying assumption:*

  Inputs are parsed enough before going further deep in execution

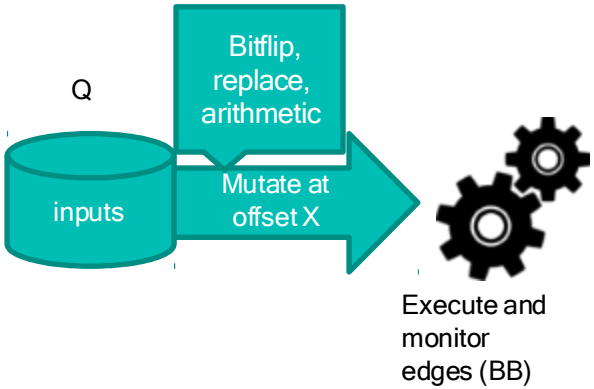
Evolving A Fuzzer



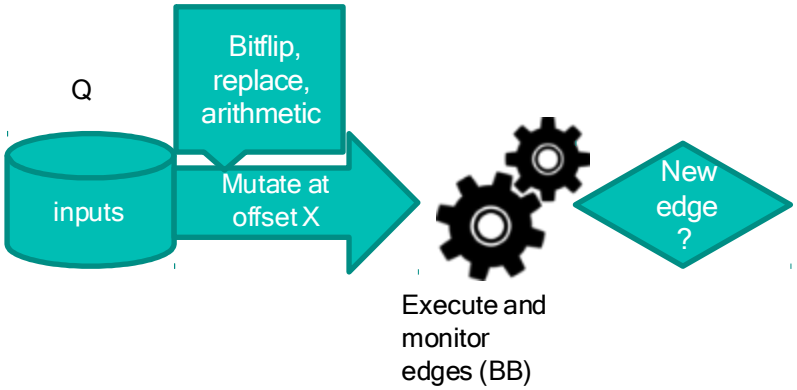
Evolving A Fuzzer



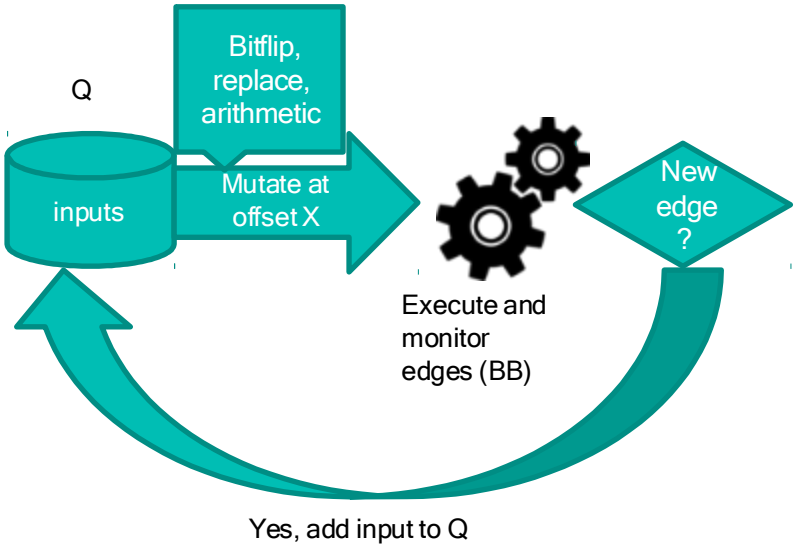
Evolving A Fuzzer



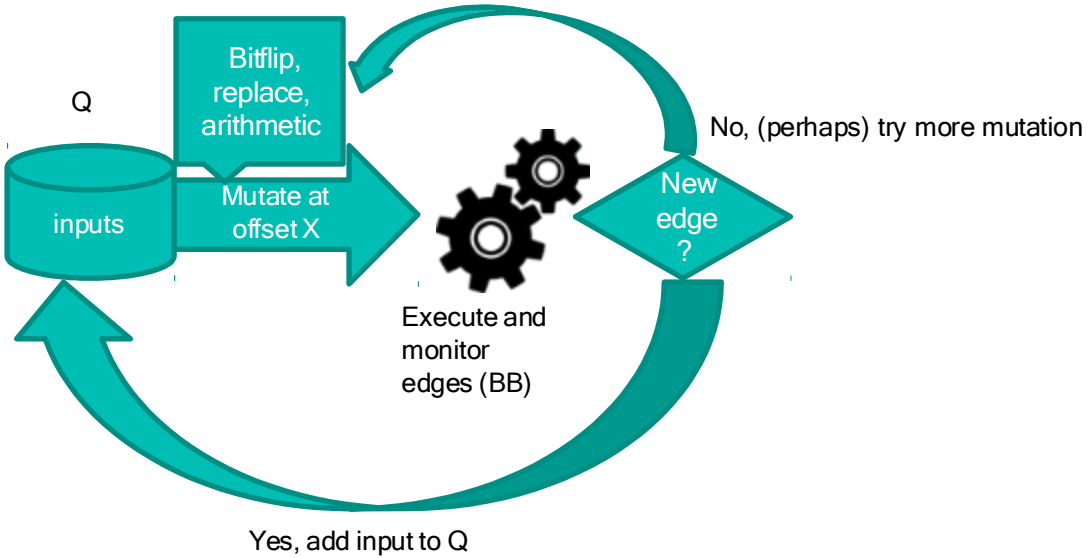
Evolving A Fuzzer



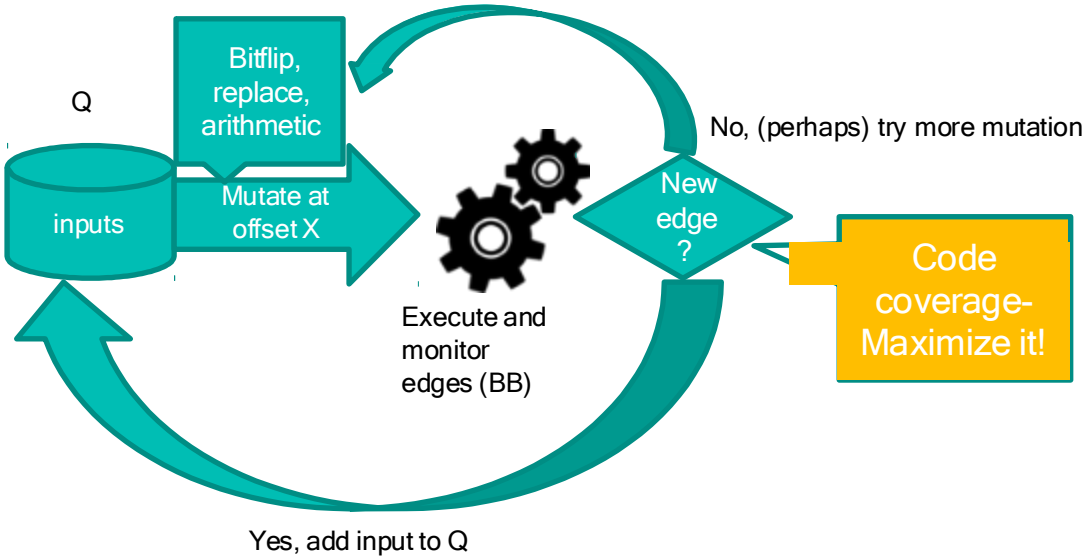
Evolving A Fuzzer



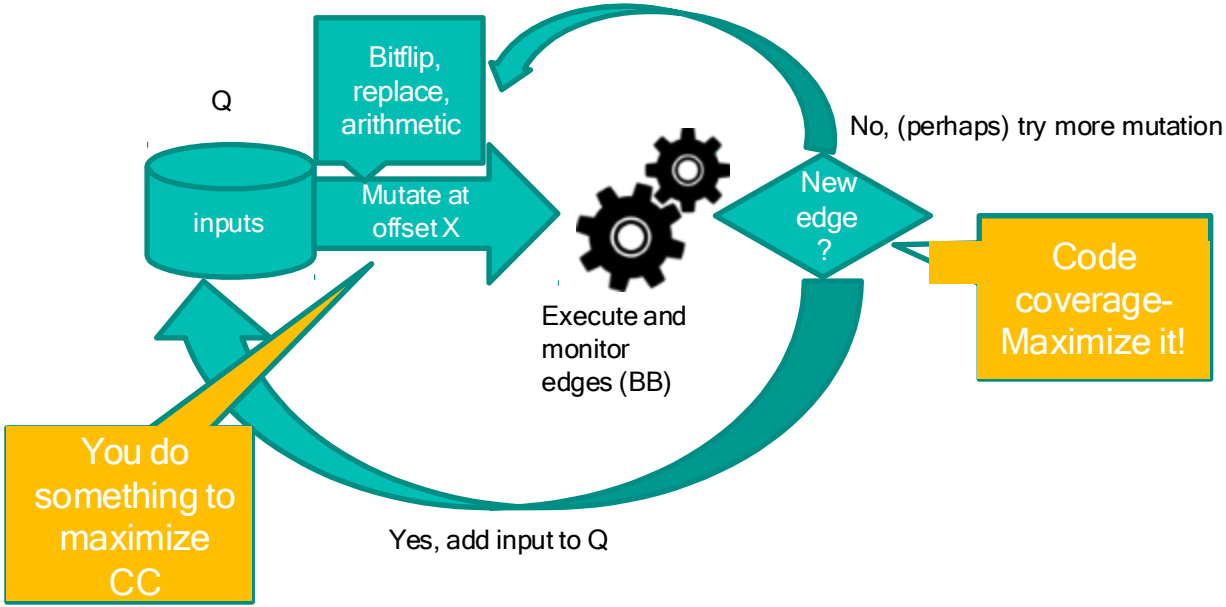
Evolving A Fuzzer



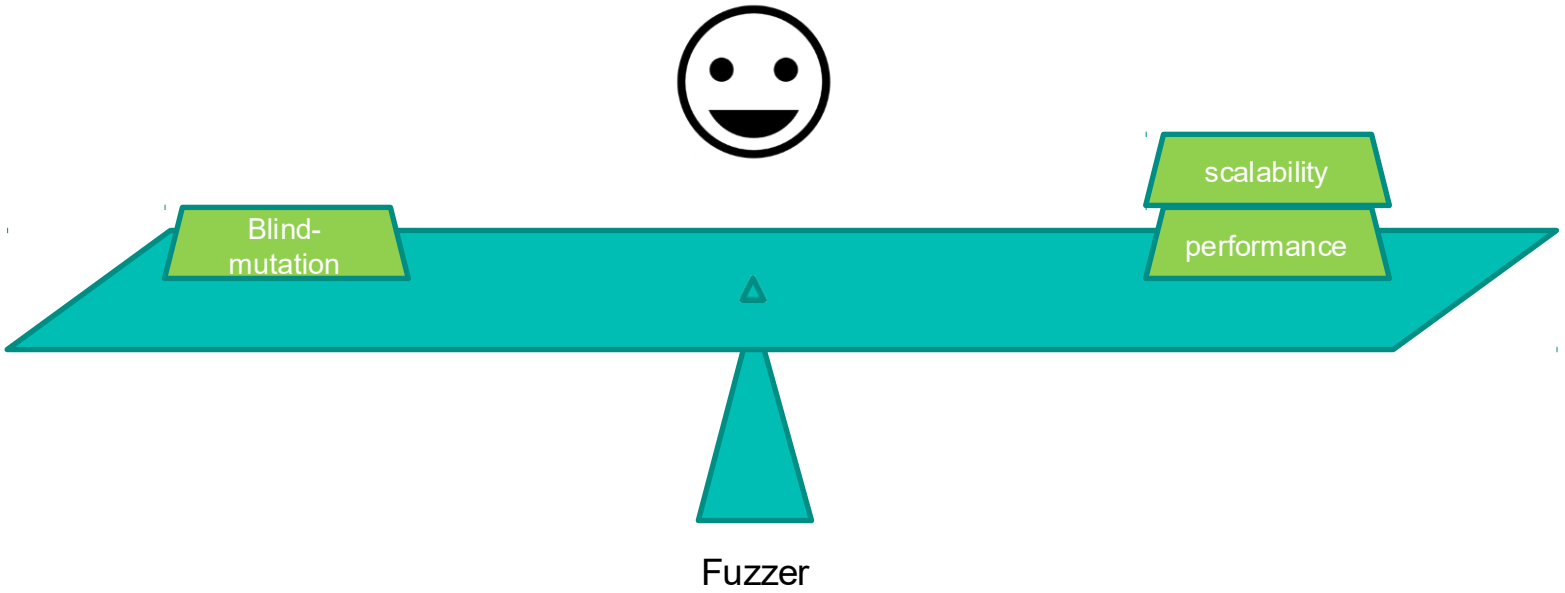
Evolving A Fuzzer



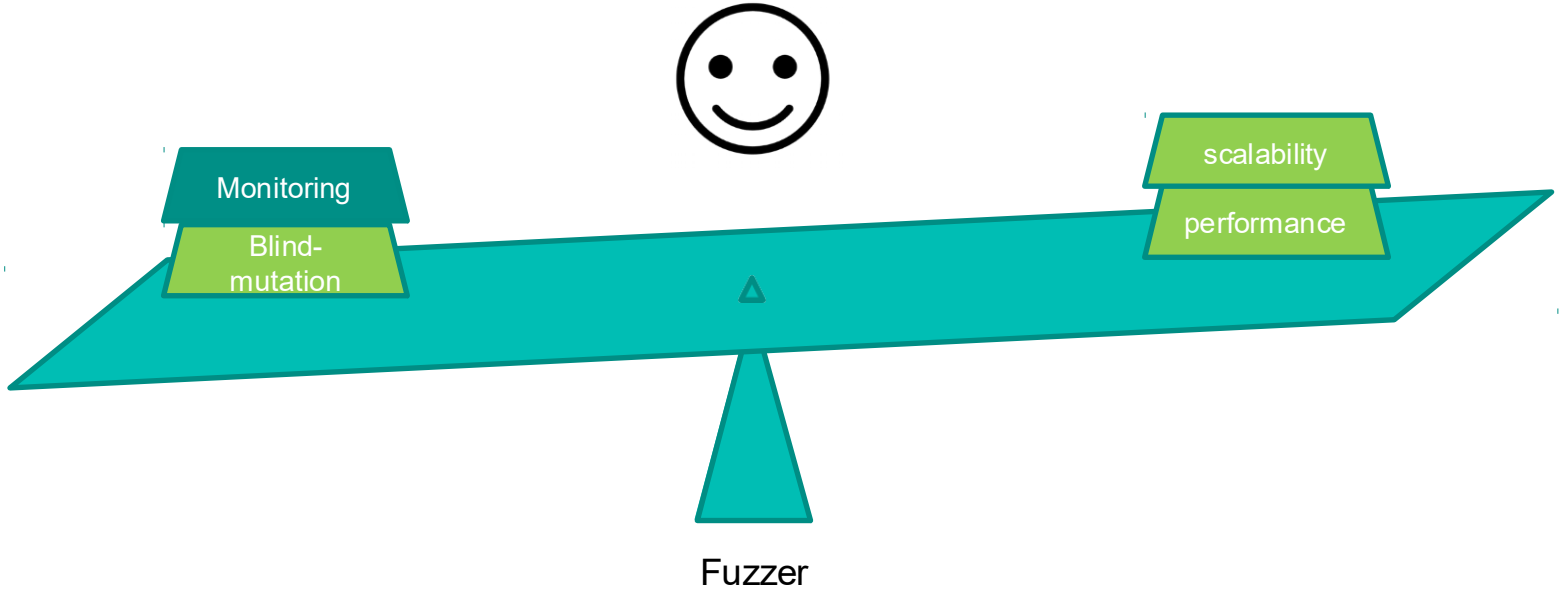
Evolving A Fuzzer



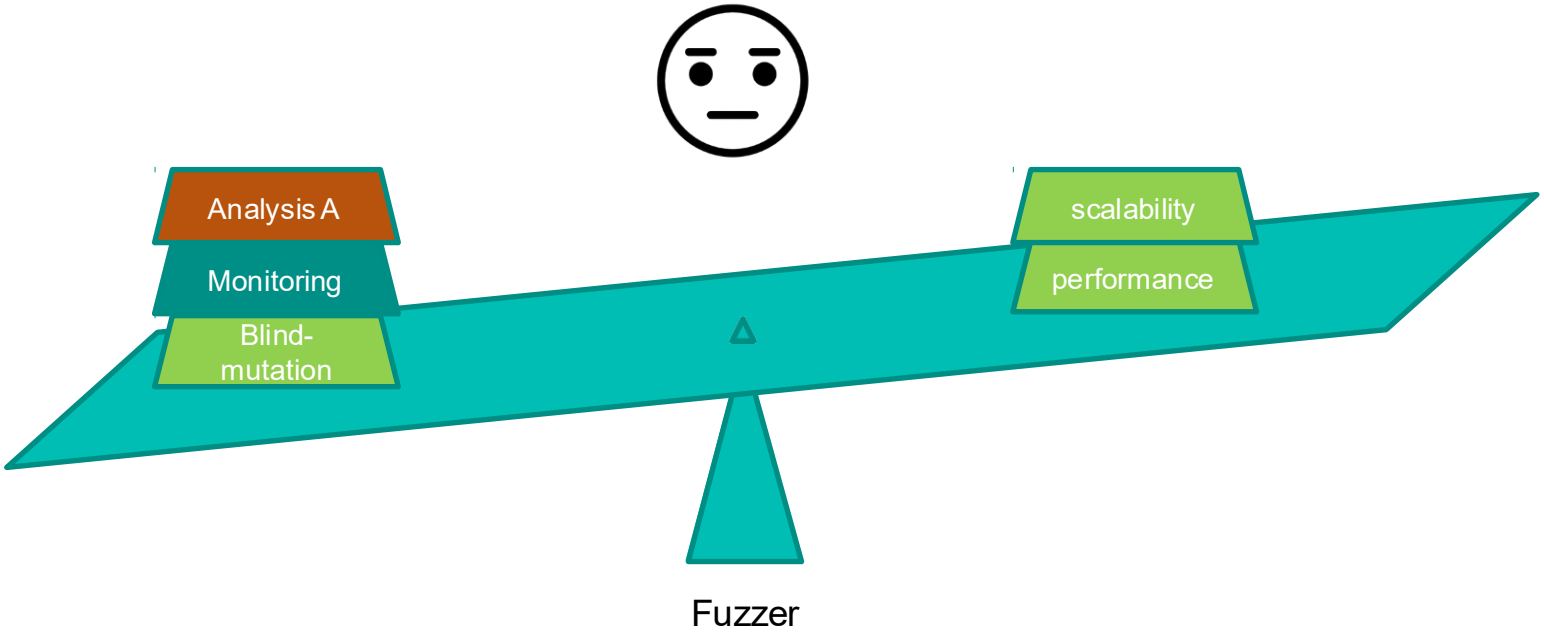
Fuzzing- A balancing Act



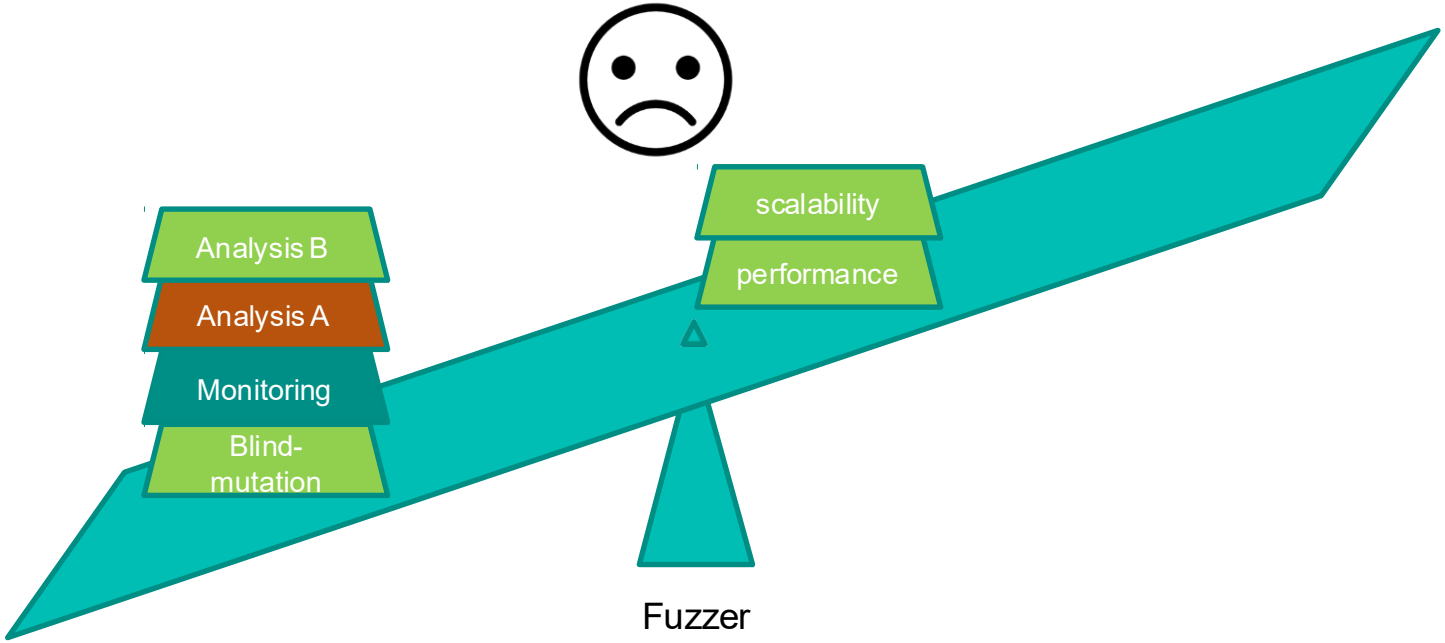
Fuzzing- A balancing Act



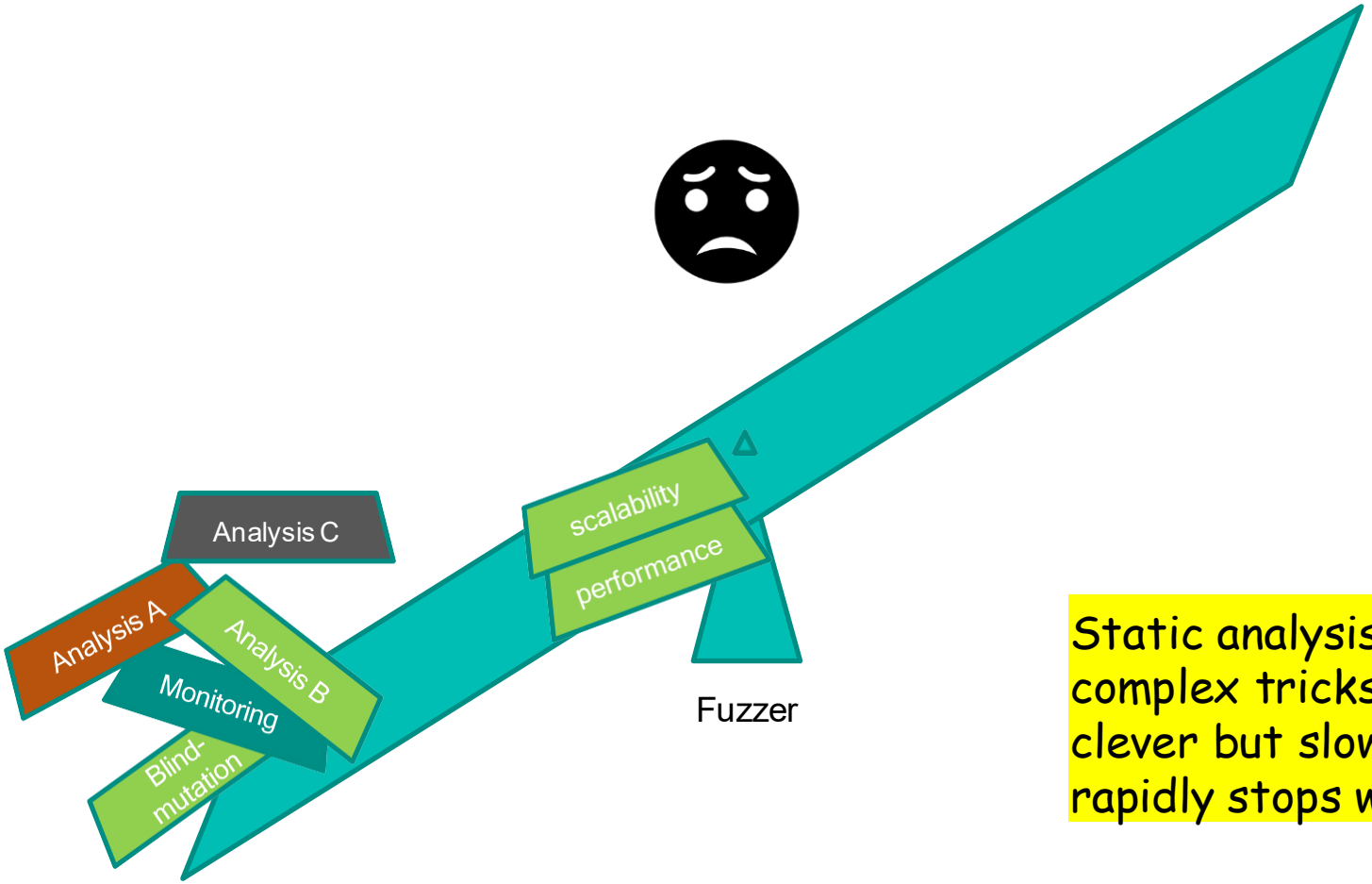
Fuzzing- A balancing Act



Fuzzing- A balancing Act

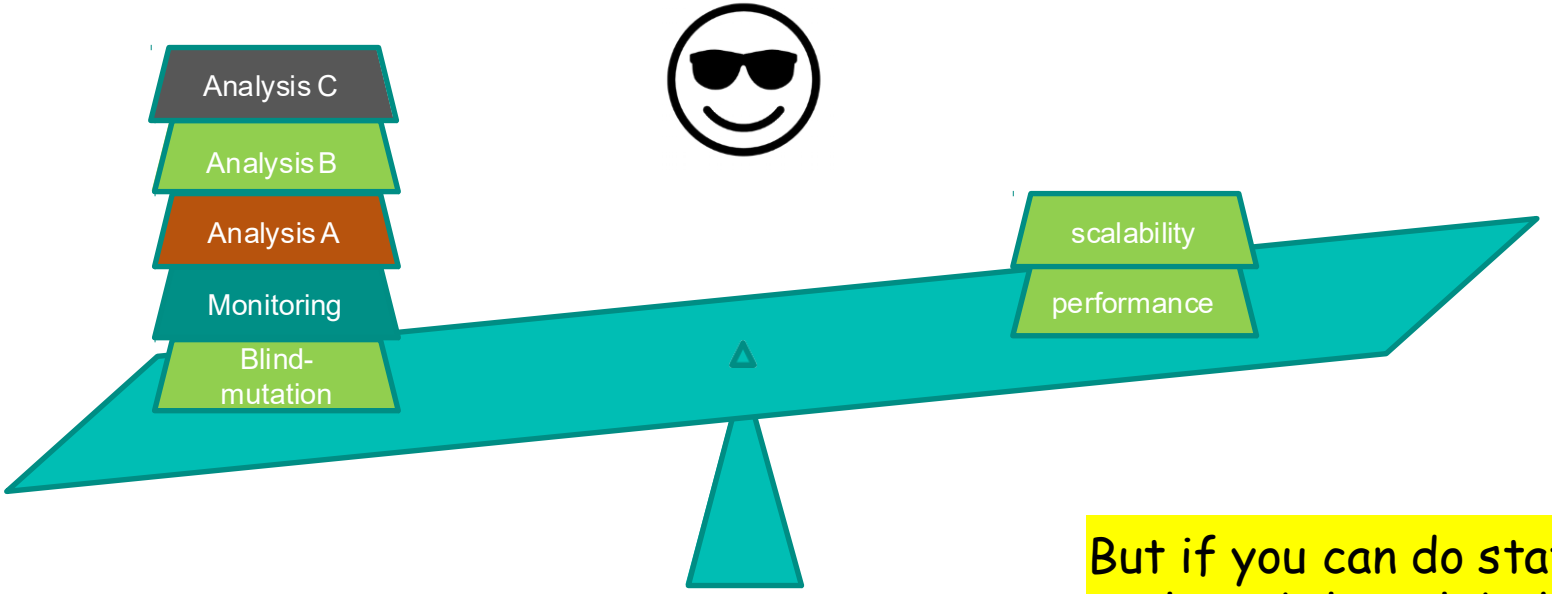


Fuzzing- A balancing Act



Static analysis and complex tricks are clever but slow and it rapidly stops working…

Fuzzing- A balancing Act



Happy Advanced Fuzzer

But if you can do static analysis *cleverly* this is the dream; but it rarely works…

Evolutionary Fuzzing

- What should be the feedback to evolve?
  - Code-coverage based fuzzing
    - Most of the contemporary fuzzers: AFL, AFLFast, Driller, VUzzer, ProbeFuzzer, CollAFL, Angora, QSYM, Nautilus, …
    - Uses code-coverage as the proxy metric for the effectiveness of a fuzzer
  - Directed fuzzing
    - Not much explored (BuzzFuzz, AFLGo, ParmeSan…)
    - There should be a way to find the destination and a sense of direction.

Evolving A Fuzzer

AFL

Open source from Google
We will use in next lab

Conclusions

- Fuzzing – seems easy unless you try it!
- Scalability and performance cannot be negotiated much!
  - A good engineering, hardware assisted monitoring
- A good place to try program analysis techniques
  - Possibility to compromise correctness to make them scalable
- Software will remain integral part of the cyber world- make it secure!

But now it'll be written by AI and it can fuzz itself… *sigh.*

I think I'm starting to get bitter.